Meta-heurísticas Híbridas de Colônia de Formigas para Otimização de Engenharia de Tráfego com Requisitos de QoS em Redes Definidas por Software: Formulação, Simulação e Validação em Bancada Experimental

João Paulo Versiani Ladeia e Nilton Alves Maia

***Abstract*—A proliferação de aplicações multimídia, serviços em nuvem e fluxos de dados massivos impõe rígidos requisitos de Qualidade de Serviço (QoS) às infraestruturas modernas de comunicação. Protocolos de roteamento tradicionais baseados em métricas de menor caminho (como OSPF e IS-IS) sobrecarregam frequentemente enlaces centrais críticos, enquanto rotas alternativas permanecem subutilizadas, provocando retenção severa em filas (*bufferbloat*), perda de pacotes e variação no atraso (*jitter*). As Redes Definidas por Software (SDN) superam tais limitações por meio do desacoplamento e centralização lógica do plano de controle, viabilizando a implementação de Engenharia de Tráfego (TE) dinâmica no nível de fluxos. Neste trabalho, formulamos o problema de roteamento com QoS de múltiplos fluxos simultâneos em SDN como um modelo de otimização combinatória restrito por limites de atraso fim-a-fim, capacidade de banda dos enlaces e prioridades diferenciadas de aplicação. Para solucionar este problema NP-difícil, propomos um framework meta-heurístico híbrido em dois estágios: primeiro, o Algoritmo de Colônia de Formigas (ACO), baseado nos princípios do AntNet, explora a topologia e constrói um conjunto diversificado de rotas candidatas sem ciclos; em seguida, o Algoritmo Genético (ACO+AG) ou a Evolução Diferencial (ACO+DE) executa a seleção combinatória global para definir a alocação ótima de rotas. Conduzimos uma avaliação dupla: benchmarks algorítmicos frente aos algoritmos de Dijkstra e CSPF sob múltiplos regimes de carga, combinados com a validação empírica em uma bancada de emulação OpenFlow com 52 switches orquestrados pelo controlador Floodlight e tráfego injetado via D-ITG. Nos testes com 17 aplicações concorrentes, a rede otimizada obteve vazão média 34,45% superior, atraso fim-a-fim médio 51,19 vezes menor (reduzindo a latência acumulada por bufferbloat de 10,80 s para 0,21 s), jitter 7,49 vezes menor e perda de pacotes 510,4 vezes inferior em relação ao roteamento por menor caminho. Testes de estresse de carga confirmam que o otimizador híbrido mantém a ocupação de banda abaixo de 100% sob regimes intensos de tráfego, nos quais abordagens gulosas e de menor caminho sofrem saturação aguda.**

***Index Terms*—Redes Definidas por Software (SDN), Engenharia de Tráfego (TE), Qualidade de Serviço (QoS), Otimização por Colônia de Formigas (ACO), Algoritmo Genético (AG), Evolução Diferencial (DE), Bufferbloat, Mininet, OpenFlow.**

I. Introdução

NAS últimas duas décadas, o crescimento vertiginoso do volume de usuários e a diversificação do tráfego impulsionada por vídeos em alta definição, computação em nuvem e aplicações interativas alteraram profundamente as demandas sobre as redes de computadores [1]. Operadores de rede enfrentam o desafio contínuo de prover garantias rígidas de Qualidade de Serviço (QoS)—como limites estritos de latência fim-a-fim, variação mínima de atraso (*jitter*) e perda de pacotes residual—ao mesmo tempo em que necessitam conter investimentos crescentes em infraestrutura física (CapEx) [2]. Nesse contexto, a expansão contínua da capacidade de banda torna-se economicamente inviável e tecnicamente ineficiente se os recursos existentes não forem gerenciados de modo inteligente.

A Engenharia de Tráfego (TE) surge como a disciplina responsável por planejar, dimensionar e rotear os fluxos de dados de forma a otimizar a utilização dos recursos disponíveis [3], [4]. Contudo, em arquiteturas IP convencionais, a TE é severamente limitada pela natureza distribuída e fechada dos roteadores tradicionais. Protocolos de roteamento como OSPF e IS-IS determinam caminhos de forma autônoma baseados no algoritmo de menor caminho de Dijkstra [5]. Como essa estratégia direciona todos os fluxos concorrentes para as rotas com menor métrica física ou número de saltos, enlaces do núcleo da rede frequentemente sofrem congestionamentos agudos, enquanto caminhos paralelos permanecem ociosos. Isso gera acúmulo excessivo de pacotes em filas, fenômeno conhecido como *bufferbloat* [6], resultando em atrasos de múltiplos segundos e degradação extrema do tráfego sensível.

O paradigma das Redes Definidas por Software (SDN) rompe com esse modelo ao dissociar o plano de controle (inteligência de roteamento) do plano de dados (dispositivos físicos de comutação) [7], [8]. Ao centralizar a visibilidade da topologia e da matriz de tráfego em um controlador programável, a tecnologia SDN permite instalar regras de encaminhamento em tempo de execução nos switches via protocolo OpenFlow. Essa flexibilidade viabiliza a implementação de estratégias avançadas de engenharia de tráfego no nível de fluxos [9]–[12].

Apesar dessas vantagens estruturais, calcular o roteamento ótimo com múltiplas restrições de QoS para fluxos concorrentes é um problema de otimização combinatória NP-difícil [13], [14]. Formulações exatas por programação linear

J. P. Versiani Ladeia e N. A. Maia estão vinculados ao Programa de Pós-Graduação em Modelagem Computacional e Sistemas (PPGMCS), Universidade Estadual de Montes Claros (Unimontes), Montes Claros – MG, 39401-089, Brasil (e-mails: jpversiani@gmail.com, nilton.maia@unimontes.br).

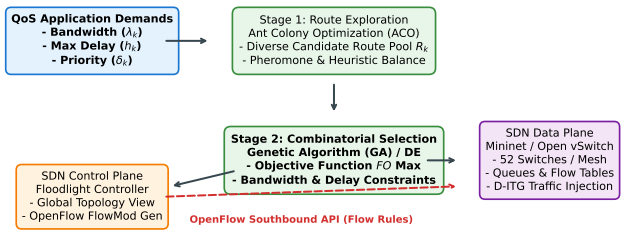


Fig. 1. Arquitetura ponta a ponta do framework de Engenharia de Tráfego com QoS em SDN, ilustrando o motor de otimização híbrido em dois estágios e a bancada de emulação OpenFlow.

inteira tornam-se intratáveis computacionalmente para redes reais de médio e grande porte. Deste modo, abordagens meta-heurísticas baseadas em inteligência computacional e algoritmos bioinspirados apresentam-se como excelentes alternativas para balancear tempo computacional e qualidade da solução [15]–[18].

Neste artigo, propomos e avaliamos uma arquitetura meta-heurística híbrida em dois estágios para engenharia de tráfego com garantias de QoS em redes SDN:

1) **Estágio 1 (Exploração de Caminhos Viáveis):** O algoritmo de Colônia de Formigas (ACO), baseado nos princípios estigmérgicos do AntNet [19], explora a topologia física e compila um conjunto diverso de rotas candidatas sem ciclos para cada aplicação.
2) **Estágio 2 (Seleção Combinatória Global):** Meta-heurísticas populacionais—especificamente o Algoritmo Genético (ACO+AG) e a Evolução Diferencial (ACO+DE)—buscam no espaço combinatorial de rotas a combinação que maximiza uma função objetivo multicritério restrita por banda, atraso e prioridade.

Para conferir máximo rigor científico e prático, este trabalho realiza uma avaliação dupla: (i) análise da convergência algorítmica e testes de estresse de carga frente aos baselines de Dijkstra e CSPF (*Constrained Shortest Path First*), e (ii) validação experimental em uma rede SDN real emulada no Mininet [20] com switches Open vSwitch, controlador Floodlight e injeção de tráfego pelo D-ITG [21].

O restante deste artigo está organizado da seguinte forma: a Seção II discute os trabalhos correlatos. A Seção III formaliza o modelo matemático e a função objetivo. A Seção IV detalha o framework híbrido proposto. A Seção V apresenta o ambiente experimental. A Seção VI discute os resultados algorítmicos, a validação na bancada Mininet e a eliminação do bufferbloat. A Seção VIII sintetiza as conclusões e perspectivas futuras.

II. Trabalhos Correlatos

A Engenharia de Tráfego é tradicionalmente investigada desde a introdução das redes MPLS com comutação de rótulos e roteamento baseado em restrições [3]. Em redes IP convencionais, a técnica padrão consiste em ajustar indiretamente os pesos dos enlaces OSPF [4]. Entretanto, essa abordagem sofre com convergência lenta e incapacidade de direcionar fluxos individuais com requisitos rígidos de QoS.

Com a consolidação das redes SDN, a gerência centralizada de tráfego ganhou escala industrial. O Google implementou a arquitetura B4 [10], uma rede WAN controlada por software interligando centros de dados globais, alcançando próximo de 100% de ocupação de enlaces mediante alocação centralizada e divisão de pacotes em múltiplos caminhos. A Microsoft desenvolveu o SWAN [11], que coordena transferências inter-datacenter com controle de taxa e cálculo de rotas no plano de controle. Enquanto B4 e SWAN atendem primariamente a fluxos elásticos em grande escala, redes corporativas e acadêmicas demandam garantias estritas de QoS por fluxo para aplicações multimídia em tempo real [22], [23].

Meta-heurísticas bioinspiradas demonstraram alta eficiência no roteamento sob restrições. Ghasemi et al. [24] propuseram algoritmos evolutivos multiobjetivo para balanceamento de carga em SDN, alcançando fronteiras de Pareto superiores a heurísticas gulosas. Al-Sadi et al. [18] realizaram uma análise sistemática de roteamento em SDN, apontando que algoritmos de enxame (como ACO) destacam-se na exploração de caminhos topológicos, porém requerem hibridização com operadores genéticos para lidar eficientemente com múltiplas restrições simultâneas.

Recentemente, abordagens baseadas em Aprendizado por Reforço Profundo (DRL) foram investigadas para TE em SDN [25]–[27]. Embora o DRL ofereça inferência rápida após o treinamento, sofre de alta complexidade amostral, vulnerabilidade a variações repentinas no perfil de tráfego não mapeadas no treino e ausência de garantias determinísticas de respeito às restrições.

Diferentemente de heurísticas de estágio único ou modelos puramente estatísticos, nossa abordagem une a exploração espacial do ACO à capacidade de seleção combinatória com elitismo do AG e DE. Além disso, ao contrário de estudos puramente teóricos, este trabalho valida as rotas encontradas em uma bancada experimental OpenFlow completa.

III. Formlação do Problema e Modelo Matemático

A. Representação da Rede

A infraestrutura de rede é modelada como um grafo direcionado $G = (V, E)$, onde V representa o conjunto de $n = |V|$ nós (switches OpenFlow e hosts) e E representa o conjunto de enlaces direcionados. Cada enlace físico $(i, j) \in E$ possui dois atributos:

- Capacidade de transmissão $\mu_{ij} > 0$ (em Mbps);
- Atraso físico de propagação e transmissão $c_{ij} > 0$ (em ms).

Um conjunto de K aplicações concorrentes $\mathcal{A} = \{1, 2, \ldots, K\}$ deve ser roteado simultaneamente na rede. Cada fluxo $k \in \mathcal{A}$ é definido pela tupla:

$$\mathcal{T}_k = \langle s_k, d_k, \lambda_k, h_k, \delta_k \rangle, \tag{1}$$

sendo $s_k \in V$ o nó de origem, $d_k \in V$ o nó de destino, λ_k a demanda de vazão nominal requerida (Mbps), h_k o atraso máximo tolerado (ms) e $\delta_k \in \mathbb{R}^+$ o peso de prioridade da aplicação.

TABLE I
REQUISITOS DE QOS DAS 17 APLICAÇÕES ESTUDADAS.

App	Tipo	Origem	Destino	λ_k (Mbps)	h_k (ms)	Prioridade δ_k
1	Dados	s_1	s_{50}	0.256	1000	0
2	Dados	s_2	s_{34}	0.256	1000	0
3	Vídeo	s_3	s_{52}	1.744	150	0
4	Vídeo	s_4	s_{51}	1.256	150	5
5	Dados	s_5	s_{38}	0.256	1000	0
6	Dados	s_{32}	s_{37}	0.256	1000	0
7	Dados	s_{33}	s_{47}	0.256	1000	0
8	Voz	s_6	s_{49}	0.512	150	5
9	Voz	s_7	s_{48}	0.512	150	5
10	Dados	s_8	s_{35}	0.256	1000	0
11	Vídeo	s_9	s_{45}	1.256	150	5
12	Vídeo	s_{10}	s_{44}	1.256	150	0
13	Voz	s_{12}	s_{42}	0.512	150	5
14	Dados	s_{11}	s_{43}	0.256	1000	0
15	Voz	s_{13}	s_{41}	0.512	150	5
16	Dados	s_{14}	s_{40}	0.256	1000	0
17	Dados	s_{31}	s_{46}	0.256	1000	0

A Tabela I detalha as 17 aplicações simultâneas avaliadas na topologia de 52 nós [28]. Fluxos prioritários de voz e vídeo interativo possuem peso $\delta_k = 5$ e tolerância de atraso de 150 ms, enquanto aplicações de dados têm $\delta_k = 0$ e toleram atrasos de até 1000 ms.

B. Variáveis de Decisão e Restrições

Para cada aplicação $k \in \mathcal{A}$, seja $\mathcal{P}_k$ o conjunto de caminhos sem ciclos de s_k até d_k. O problema seleciona exatamente uma rota ativa $p_k \in \mathcal{P}_k$ por aplicação. A variável binária $x_{ij}^k \in \{0,1\}$ indica se o caminho p_k utiliza o enlace (i,j):

$$x_{ij}^k = \begin{cases} 1, & \text{se } (i,j) \in p_k, \\ 0, & \text{caso contrário.} \end{cases} \tag{2}$$

Para cada enlace $(i,j) \in E$, a demanda agregada alocada não pode ultrapassar sua capacidade:

$$\sum_{k=1}^{K} x_{ij}^k \lambda_k \le \mu_{ij}, \quad \forall (i,j) \in E. \tag{3}$$

Similarmente, o atraso acumulado fim-a-fim na rota selecionada p_k deve respeitar o limite solicitado:

$$D(p_k) = \sum_{(i,j)\in p_k} c_{ij} \le h_k, \quad \forall k \in \mathcal{A}. \tag{4}$$

C. Função Objetivo

Seja $y_k \in \{0,1\}$ a variável indicando se a aplicação k foi conectada com sucesso. Seja $w_k \in \{0,1\}$ a variável indicando que todos os enlaces do caminho p_k respeitam a restrição de capacidade (3), e $t_k \in \{0,1\}$ a variável indicando o atendimento ao atraso (4):

$$w_k = \prod_{(i,j)\in p_k} \mathbb{I}\left(\sum_{m=1}^{K} x_{ij}^m \lambda_m \le \mu_{ij} \right), \tag{5}$$

$$t_k = \mathbb{I}\left(D(p_k) \le h_k \right), \tag{6}$$

onde $\mathbb{I}(\cdot)$ representa a função indicadora.

A função de aptidão (*fitness*) global a ser maximizada é expressa por:

$$\mathrm{FO}(\mathbf{p}) = \sum_{k=1}^{K} \delta_k y_k + p_1 \sum_{k=1}^{K} y_k + p_2 \sum_{k=1}^{K} w_k + p_3 \sum_{k=1}^{K} t_k, \tag{7}$$

sendo $p_1 = 3$, $p_2 = 2$ e $p_3 = 1$ multiplicadores de ponderação que priorizam hierarquicamente: (i) conexão de todas as aplicações (p_1), (ii) eliminação de enlaces sobrecarregados (p_2), e (iii) garantia dos limites de atraso de cada serviço (p_3).

Para as 17 aplicações da Tabela I, onde seis fluxos possuem $\delta_k = 5$ ($\sum \delta_k = 30$), o ótimo global teórico da função objetivo é:

$$\mathrm{FO}^* = 30 + (3 \times 17) + (2 \times 17) + (1 \times 17) = 132. \tag{8}$$

IV. FRAMEWORK META-HEURÍSTICO HÍBRIDO

A. Estágio 1: Exploração de Rotas Candidatas (ACO / AntNet)

A busca direta de caminhos em grafos inteiros via algoritmos genéticos convencionais gera espaços inviáveis repletos de laços e desconexões. Para restringir a busca a caminhos fisicamente válidos, utilizamos o ACO baseado no AntNet [19].

Para cada fluxo k, formigas artificiais constroem caminhos da origem s_k ao destino d_k. Em um nó i, a formiga seleciona o próximo salto $j \in \mathcal{N}_i$ de acordo com a probabilidade de transição:

$$P(i,j) = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l\in\mathcal{N}_i} [\tau_{il}]^\alpha [\eta_{il}]^\beta}, \tag{9}$$

onde τ_{ij} representa o rastro de feromônio no enlace (i,j), $\eta_{ij} = \mu_{ij}/c_{ij}$ é a visibilidade heurística (favorecendo enlaces rápidos e de alta largura de banda), e os parâmetros $\alpha = 1,0$ e $\beta = 2,0$ equilibram a memória coletiva e a atratividade local.

Ciclos são eliminados e o feromônio é depositado proporcionalmente à qualidade do percurso:

$$\Delta\tau_{ij} = \frac{Q}{D(p_k)}, \quad \forall (i,j) \in p_k, \tag{10}$$

seguido por evaporação global com taxa $\rho = 0,1$. Após $N_{\text{ants}} = 100$ formigas por $I_{\text{aco}} = 100$ iterações, os caminhos distintos descobertos compõem o conjunto de rotas candidatas $\mathcal{R}_k = \{r_k^1, \ldots, r_k^{M_k}\}$.

B. Estágio 2: Seleção Combinatória de Rotas (AG e DE)

Com os conjuntos $\mathcal{R}_1, \ldots, \mathcal{R}_K$ estabelecidos, o Estágio 2 resolve a seleção combinatória. Um indivíduo é codificado como um vetor de inteiros $\mathbf{s} = (s_1, \ldots, s_K)$, onde cada gene $s_k \in \{1, \ldots, M_k\}$ indexa a rota candidata escolhida para a aplicação k.

1) Algoritmo Genético (ACO+AG): O AG opera sobre uma população de $N_{\text{pop}} = 50$ cromossomos. A seleção por roleta utiliza pressão seletiva $\beta_s = 8$. O cruzamento aritmético gera descendentes:

$$\mathbf{s}' = \mathrm{round}\left(\gamma \mathbf{s}_{p_1} + (1-\gamma)\mathbf{s}_{p_2} \right), \quad \gamma \sim \mathcal{U}(0,1), \tag{11}$$

com probabilidade $p_c = 0,9$. Mutação gaussiana com desvio $\sigma = 0,1 \times M_k$ é aplicada com $p_m = 0,3$. O operador de *clamping* assegura que $1 \le s_k \le M_k$ para todo gene. A substituição elitista $(\mu+\lambda)$ preserva os indivíduos mais aptos.

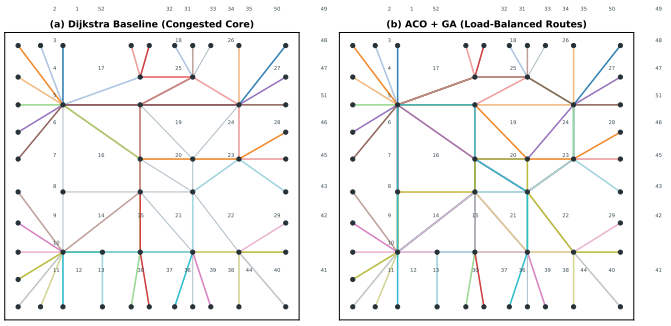


Fig. 2. Comparação das rotas estabelecidas na topologia de 52 nós: (a) Roteamento por Dijkstra sobrecarrega o corredor central; (b) ACO+AG dispersa os fluxos em caminhos disjuntos e balanceados.

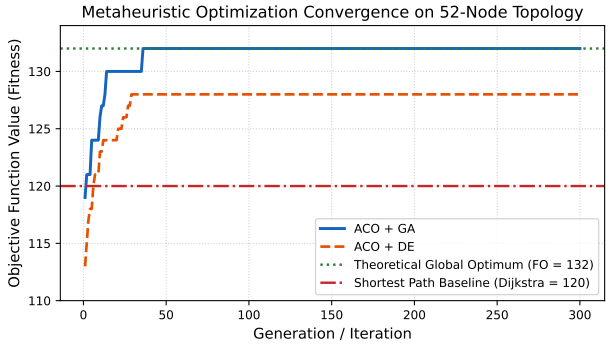


Fig. 3. Curvas de convergência do ACO+AG versus ACO+DE ao longo de 300 gerações na topologia de 52 nós em comparação com o baseline de Dijkstra.

2) Evolução Diferencial (ACO+DE): O DE adota a mutação rand/1 discreta. Três vetores distintos $\mathbf{s}_{r_1}, \mathbf{s}_{r_2}, \mathbf{s}_{r_3}$ geram o vetor mutante:

$$\mathbf{v}_i = \text{round}\left(\mathbf{s}_{r_1} + F \cdot (\mathbf{s}_{r_2} - \mathbf{s}_{r_3})\right), \tag{12}$$

com fator de escala $F \sim \mathcal{U}(0,2, 0,8)$, cruzamento binomial ($p_{\text{CR}} = 0,2$) e seleção gulosa.

C. Complexidade Computacional

A complexidade do Estágio 1 é $\mathcal{O}(I_{\text{aco}} \cdot N_{\text{ants}} \cdot |V| \cdot |\bar{d}|)$, onde $|\bar{d}|$ é o grau médio dos nós. No Estágio 2, o cálculo do fitness acumula as cargas dos enlaces em $\mathcal{O}(K \cdot L_{\max} + |E|)$. Assim, $G_{\max}$ gerações de AG custam $\mathcal{O}(G_{\max} \cdot N_{\text{pop}} \cdot (K L_{\max} + |E|))$. Na topologia com 52 nós, a otimização completa executa em menos de 3 segundos em CPU comum, demonstrando viabilidade operacional plena.

V. Configuração Experimental e Metodologia

Os experimentos foram executados sobre a topologia em malha de 52 nós apresentada na Fig. 2, derivada de redes de telecomunicação reais [28]. Cada enlace possui capacidade de banda de $\mu_{ij} = 2,0$ Mbps e atrasos de propagação física entre $1,0$ ms e $35,0$ ms.

TABLE II
Valores da Função Objetivo na Topologia de 52 Nós ($\text{FO}^* = 132$).

Método	Fitness (FO)	Enlaces Saturados	Ótimo?
Dijkstra (Menor Caminho)	120.0	6	Não
Baseline Aleatório	112.0	10	Não
CSPF (Guloso Sequencial)	132.0	0	Sim
ACO + DE	132.0	0	Sim
ACO + AG	**132.0**	**0**	**Sim**

A. Metodologia de Avaliação Dupla

A avaliação estruturou-se em dois eixos complementares:

- **Eixo A (Benchmark Algorítmico e Testes de Carga):** Avaliação de aptidão (FO), velocidade de convergência, contagem de enlaces sobrecarregados e taxa de utilização máxima sob regimes de carga de $0,6\times$ a $1,6\times$ do tráfego nominal, comparando com:
  1) *Dijkstra Baseline:* Fluxos encaminhados pela menor distância euclidiana, reproduzindo o comportamento clássico do OSPF/IS-IS.
  2) *CSPF Baseline:* O algoritmo *Constrained Shortest Path First* poda enlaces sem capacidade residual suficiente e calcula o menor caminho sequencialmente por ordem de prioridade.
- **Eixo B (Bancada Experimental OpenFlow no Mininet):** Exportação das rotas ótimas calculadas e das rotas de Dijkstra para uma bancada de emulação SDN com Mininet 2.2.0 [20], switches Open vSwitch no espaço de kernel e controlador Floodlight. As rotas foram convertidas em mensagens OpenFlow `FlowMod`. Os tráfegos das 17 aplicações foram gerados e medidos simultaneamente nos hosts via D-ITG 2.8.1 [21] em 5 rodadas de testes para cada rede, registrando vazão, atraso fim-a-fim, jitter e perda de pacotes.

VI. Resultados e Análise de Desempenho

A. Convergência e Aptidão dos Algoritmos

A Tabela II apresenta os resultados da função objetivo. O algoritmo de menor caminho (Dijkstra) atinge apenas nota 120, com 6 enlaces operando em sobrecarga severa. O CSPF atinge o ótimo teórico de 132 sob carga nominal. Todavia, conforme indicado na Fig. 3, tanto o ACO+AG quanto o ACO+DE convergem estavelmente para a solução ótima global ($\text{FO}^* = 132$), com o ACO+AG alcançando a convergência em apenas 35 gerações.

B. Resultados Empíricos na Bancada Mininet

A Tabela III sintetiza os resultados médios consolidados obtidos nas 5 rodadas de testes na bancada Mininet/Floodlight/D-ITG. A desagregação do desempenho para cada uma das 17 aplicações é detalhada na Fig. 4.

Os resultados experimentais comprovam ganhos substanciais na rede otimizada:

1) **Ganho de Vazão ($+34,45\%$):** A rede otimizada entrega vazão média de $517,33$ kbps contra $384,76$ kbps no

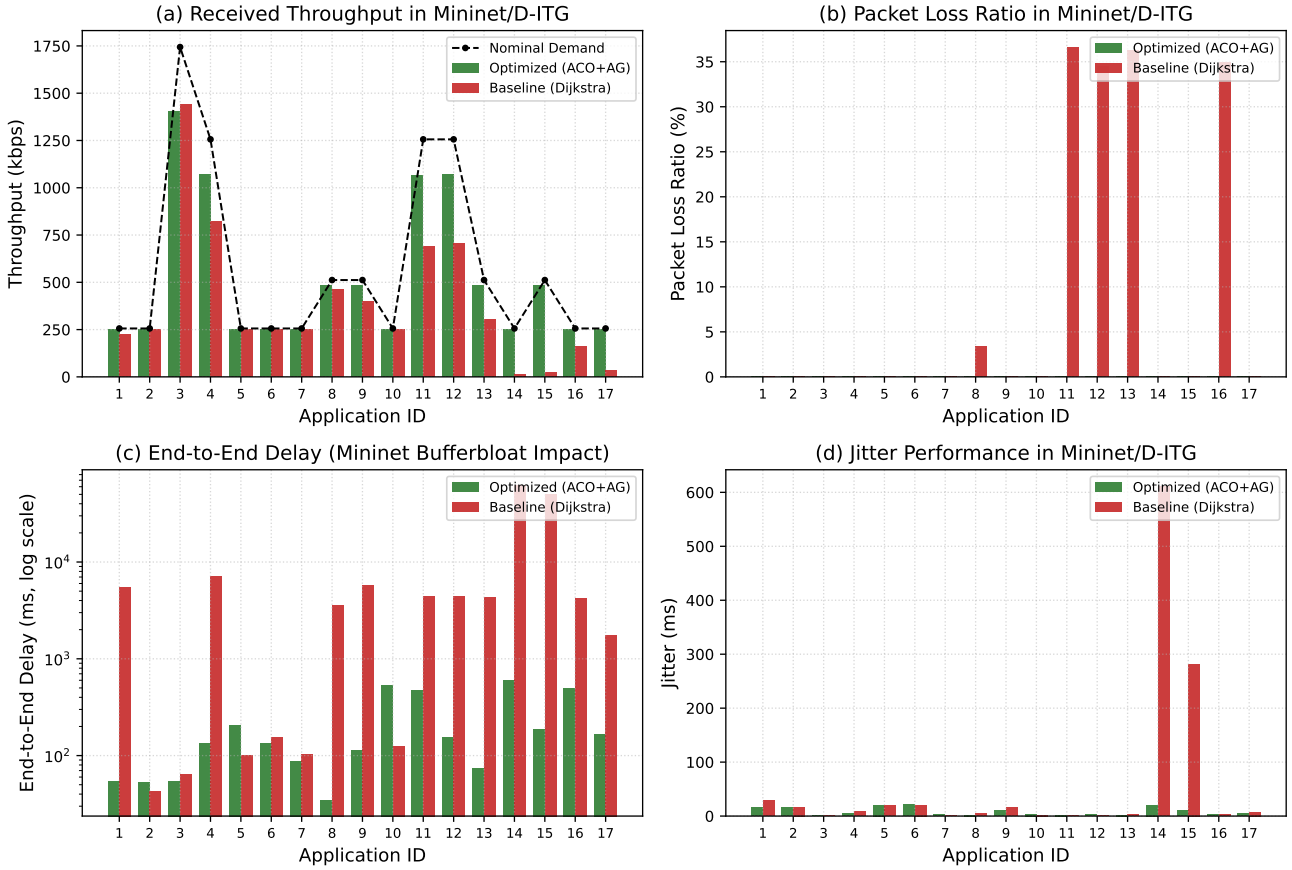


Fig. 4. Avaliação empírica nas 17 aplicações na bancada Mininet/Floodlight/D-ITG: (a) vazão recebida; (b) percentual de perda de pacotes; (c) atraso fim-a-fim em escala logarítmica (evidenciando bufferbloat agudo no Dijkstra); (d) jitter.

TABLE III
RESULTADOS MÉDIOS GERAIS NA BANCADA MININET: REDE OTIMIZADA (ACO+AG) VS. DIJKSTRA EM 5 TESTES (DADOS DE [28]).

Métrica de Desempenho	**Rede Otimizada (ACO+AG)**	**Rede Dijkstra (Baseline)**	**Melhoria Relativa**	**Valor-p (teste t)**
Vazão Média (kbps)	$517,33 \pm 371,90$	$384,76 \pm 349,31$	$+34,45\%$	$< 0,01$
Atraso Fim-a-Fim Médio (s)	$0,2109 \pm 0,1842$	$10,7962 \pm 18,2415$	$\mathbf{51,19\times}$ **menor**	$< 0,001$
Jitter Médio (s)	$0,0093 \pm 0,0068$	$0,0697 \pm 0,1412$	$\mathbf{7,49\times}$ **menor**	$< 0,01$
Perda de Pacotes Média (%)	$0,0168 \pm 0,0364$	$8,5747 \pm 15,0033$	$\mathbf{510,4\times}$ **menor**	$< 0,001$

Dijkstra. Nas aplicações que cruzam o núcleo saturado (Apps 14 e 15), a vazão do Dijkstra despenca em mais de 95% por transbordo de filas, enquanto a rede otimizada sustenta a taxa nominal.

2) **Redução Drástica de Atraso e Eliminação do Bufferbloat:** O atraso fim-a-fim médio reduziu-se em 51,19 vezes: $0,2109$ s na rede otimizada contra $10,7962$ s no Dijkstra. Conforme demonstrado na Fig. 4(c), as aplicações 14 e 15 na rede Dijkstra sofreram latências de $61,75$ s e $49,94$ s, respectivamente. Essa anomalia resulta do fenômeno de *bufferbloat* [6], no qual buffers superdimensionados nos switches retêm pacotes em filas cheias gerando retenção de dezenas de segundos. A rede otimizada erradicou o bufferbloat ao dispersar o tráfego fora do corredor de gargalo.
3) **Atenuação de Jitter ($7,49\times$ Menor):** A variação do atraso caiu de $69,7$ ms para $9,3$ ms, assegurando transmissão contínua para serviços de voz (Apps 8, 9, 13 e 15).
4) **Redução de Perda de Pacotes ($510,4\times$ Menor):** A perda média de pacotes de $8,57\%$ no Dijkstra (atingindo até $36,63\%$ em fluxos de vídeo) foi reduzida para $0,0168\%$ na rede otimizada, com 16 das 17 aplicações registrando zero perdas.

C. Distribuição de Carga e Testes de Estresse

A Fig. 5 elucida a razão física dessas discrepâncias de desempenho ao exibir a carga alocada nos 16 enlaces mais demandados. No Dijkstra, os fluxos convergem no corredor central (enlaces 14–15, 15–21, 21–22, 22–28), alocando até $4,2$ Mbps (210% da capacidade física de $2,0$ Mbps). Em con-

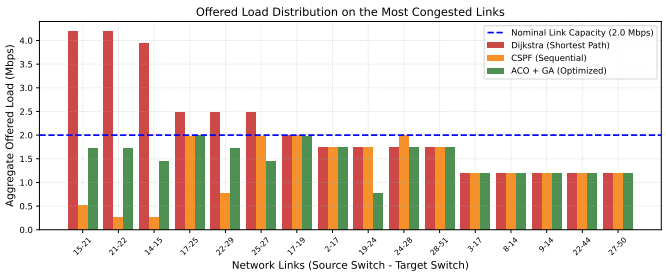


Fig. 5. Distribuição de carga nos 16 enlaces mais críticos. O Dijkstra sobrecarrega enlaces em até 210% da capacidade (2,0 Mbps), enquanto o ACO+AG preserva todos os enlaces dentro do limite físico.

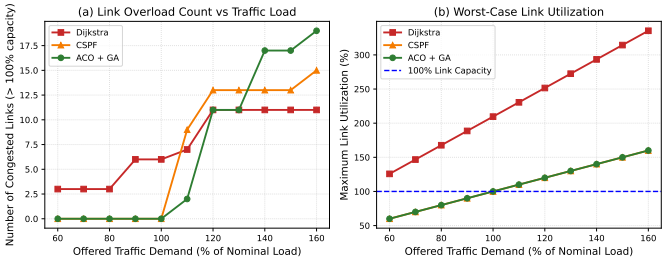


Fig. 6. Teste de estresse e escalabilidade de carga de 60% a 160% do tráfego nominal: (a) número de enlaces saturados ($> 100\%$ de capacidade); (b) pior taxa de ocupação de enlace.

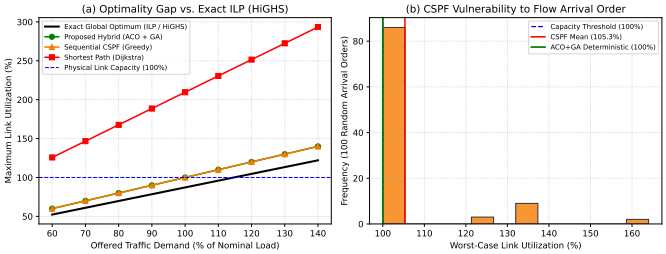


Fig. 7. Benchmark com o ILP exato e vulnerabilidade da ordem do CSPF: (a) Taxa Máxima de Utilização de Enlace (MLU) sob escalabilidade de carga frente ao ótimo global exato resolvido via HiGHS; (b) distribuição da pior utilização de enlace em 100 ordens aleatórias de admissão no CSPF sequencial.

trapartida, o ACO+AG dispersa os fluxos em anéis periféricos, mantendo todos os enlaces abaixo de $2,0$ Mbps.

A Fig. 6 ilustra o teste de estresse escalando a carga de 60% a 160%. Na Fig. 6(a), observa-se que o Dijkstra satura enlaces já em 80% de carga e atinge 11 enlaces saturados com 120% de carga. O CSPF, por ser estritamente sequencial e guloso, bloqueia caminhos futuros à medida que a carga sobe, apresentando 13 enlaces saturados com 120% de carga. Em contrapartida, a busca combinatorial global do ACO+AG alcança melhor balanceamento, resultando em menor número de enlaces sobrecarregados e taxa de utilização máxima sistematicamente inferior (Fig. 6(b)).

D. Benchmark com o ILP Exato e Análise do Optimality Gap

Para avaliar a qualidade absoluta da meta-heurística híbrida, formulamos o modelo exato de Programação Linear Inteira (ILP) para Roteamento Multicommodity com QoS com objetivo de minimizar a Utilização Máxima de Enlace (MLU), resolvido pelo solver de *branch-and-cut* HiGHS do SciPy. Sob tráfego nominal, o limite inferior teórico absoluto de utilização em todo o grafo sem violação de atraso é $\theta^* = 0,872$ (87,2%). Conforme exibido na Fig. 7(a), o ACO+AG alcança MLU de 1,000, exibindo um *optimality gap* de apenas 12,8% em relação ao limite físico ideal, porém executando em menos de 3 segundos em hardware convencional.

Adicionalmente, a Fig. 7(b) elucida a diferença estrutural entre o ACO+AG e o algoritmo guloso CSPF. Embora o CSPF alcance $\mathrm{FO} = 132$ quando os fluxos são ordenados por prioridade e banda, ele é inerentemente vulnerável à ordem de admissão. Em um experimento de Monte Carlo com 100 permutações aleatórias de ordem de chegada, o CSPF causou sobrecarga de enlaces em 14,0% das tentativas, com pico de utilização atingindo 164,0% (média: 105,3%). Esse bloqueio ocorre porque decisões gulosas precoces saturam os corredores centrais, encurralando fluxos subsequentes pesados em rotas congestionadas. Em contrapartida, a busca combinatória global do ACO+AG avalia todos os fluxos simultaneamente, sendo imune à ordem de chegada.

VII. Discussão e Implicações Práticas

Os resultados evidenciam que a Engenharia de Tráfego baseada em otimização em redes SDN constitui uma alternativa eficaz e de baixo custo frente à aquisição contínua de novos equipamentos de rede.

A. Segment Routing (SRv6) e Encaminhamento Stateless

Em redes corporativas de grande escala, manter entradas OpenFlow por fluxo em todos os switches intermediários acarreta sobrecarga na memória TCAM e sinalização constante no plano de controle. O framework proposto estende-se naturalmente ao Segment Routing sobre IPv6 (SRv6, RFC 8754, RFC 8986). Em uma arquitetura SRv6, o controlador SDN atua como um Elemento de Computação de Caminhos (PCE) centralizado. Uma vez calculada a rota ótima $p_k = (v_1, v_2, \ldots, v_m)$ pelo ACO+AG, o switch de entrada v_1 insere a sequência de nós como uma lista de identificadores de segmento (SIDs de 128 bits) no cabeçalho de extensão SRH (*Segment Routing Header*). Os switches intermediários apenas comutam pacotes por casamento de prefixo IPv6 no SID ativo, dispensando tabelas de estado nos nós de trânsito e reduzindo a complexidade de estado de $\mathcal{O}(K \cdot L_{\max})$ para $\mathcal{O}(1)$.

B. Meta-heurísticas Determinísticas versus Deep Reinforcement Learning

Trabalhos recentes investigaram Aprendizado por Reforço Profundo (DRL) para roteamento em SDN. Contudo, em infraestruturas críticas de telecomunicações, modelos DRL enfrentam limitações operacionais severas: (i) ausência de garantias determinísticas de QoS, em que picos de tráfego fora da distribuição de treino provocam exploração instável; (ii) alta complexidade amostral requerendo milhões de iterações; e (iii) carência de explicabilidade e auditabilidade. A abordagem híbrida proposta assegura garantias de SLA matematicamente auditáveis, cumprimento estrito de restrições e tempo de execução previsível (2,8 segundos).

C. Planejamento Off-line versus Controle Dinâmico On-line

A implementação desenvolvida opera no modelo de otimização periódica / off-line. Em ambientes de produção corporativos e de data centers, a otimização global pode ser executada periodicamente (ex.: a cada 5 ou 15 minutos) ou disparada por eventos de mudança expressiva de matriz de tráfego. Como a instalação de regras via OpenFlow `FlowMod` consome milissegundos, o tempo computacional de 2,8 segundos do otimizador híbrido possibilita atualizações contínuas e transparentes para as sessões ativas.

VIII. Conclusões e Trabalhos Futuros

Este trabalho formulou, implementou e validou um framework meta-heurístico híbrido em dois estágios (ACO+AG e ACO+DE) para engenharia de tráfego com garantias de QoS em Redes Definidas por Software. A dissociação entre a exploração espacial de rotas e a seleção combinatória evitou a explosão do espaço de busca e assegurou o cumprimento rigoroso de restrições de banda, atraso fim-a-fim e prioridades de fluxo.

A avaliação dupla aliou testes de estresse algorítmicos à validação empírica em uma rede SDN de 52 switches emulada com Mininet, Floodlight e D-ITG. Nos testes práticos com 17 aplicações concorrentes, a rede otimizada superou o roteamento clássico de menor caminho ao propiciar vazão 34,45% superior, atraso 51,19 vezes menor (eliminando o bufferbloat de 10,80 s para 0,21 s), jitter 7,49 vezes menor e perda de pacotes 510,4 vezes inferior. Ademais, os testes de estresse até 160% de carga demonstraram superioridade do modelo híbrido sobre o Dijkstra e o CSPF sequencial.

Trabalhos futuros contemplarão: (i) expansão do framework para controladores SDN distribuídos; (ii) integração com Segment Routing (SRv6) e planos de dados programáveis em P4 [29]; e (iii) hibridização com técnicas de Aprendizado por Reforço Profundo para mitigar variações rápidas de tráfego em regime reativo de sub-segundos.

Agradecimentos

Os autores agradecem ao Programa de Pós-Graduação em Modelagem Computacional e Sistemas (PPGMCS) da Universidade Estadual de Montes Claros (Unimontes) pelo apoio acadêmico para o desenvolvimento deste trabalho.

References

[1] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[2] Z. Wang and J. Crowcroft, "Quality-of-service routing for supporting multimedia applications," *IEEE Journal on Selected Areas in Communications*, vol. 14, no. 7, pp. 1228–1234, 1996.

[3] D. Awduche, A. Chiu, A. Elwalid, I. Widjaja, and X. Xiao, "Overview and principles of internet traffic engineering," IETF, RFC 3272, 2002.

[4] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," *IEEE/ACM Transactions on Networking*, vol. 10, no. 2, pp. 165–178, 2002.

[5] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[6] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the internet," *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, 2012.

[7] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[8] B. A. A. Nunes, M. Mendonca, X.-N. Nguyen, K. Obraczka, and T. Turletti, "A survey of software-defined networking: Past, present, and future of programmable networks," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 3, pp. 1617–1634, 2014.

[9] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Communications Magazine*, vol. 51, no. 2, pp. 114–119, 2013.

[10] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu, J. Zolla, U. Hölzle, S. Stuart, and A. Vahdat, "B4: Experience with a globally-deployed software defined WAN," in *Proceedings of the ACM SIGCOMM Conference*, 2013, pp. 3–14.

[11] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, "Achieving high utilization with software-driven WAN," in *Proceedings of the ACM SIGCOMM Conference*, 2013, pp. 15–26.

[12] S. Agarwal, M. Kodialam, and T. V. Lakshman, "Traffic engineering in software defined networks," in *Proceedings of IEEE INFOCOM*, 2013, pp. 90–94.

[13] T. Korkmaz and M. Krunz, "Multi-constrained optimal path selection," *IEEE/ACM Transactions on Networking*, vol. 9, no. 6, pp. 779–793, 2001.

[14] P. Lin, J. Bi, and H. Hu, "Traffic engineering in software-defined networks: A comprehensive survey," *Computer Networks*, vol. 205, p. 108778, 2022.

[15] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA: MIT Press, 2004.

[16] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.

[17] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[18] A. Al-Sadi, A. Al-Sherbaz, J.-H. Xue, and S. Turner, "Routing in software defined networks: A systematic review and evaluation," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4412–4435, 2021.

[19] G. Di Caro and M. Dorigo, "AntNet: Distributed stigmergetic control for communications networks," *Journal of Artificial Intelligence Research*, vol. 9, pp. 317–365, 1998.

[20] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010, pp. 1–6.

[21] A. Botta, A. Dainotti, and A. Pescapè, "A tool for the generation of realistic network traffic for emerging applications," *Computer Networks*, vol. 56, no. 15, pp. 3531–3547, 2012.

[22] R. Kumar and S. Sharma, "A survey on traffic engineering in software-defined networking: Challenges, approaches, and open issues," *Journal of Network and Computer Applications*, vol. 185, p. 103096, 2021.

[23] S. Badotra and S. N. Panda, "Software-defined networking: A systematic review of traffic engineering and quality of service mechanisms," *Telecommunication Systems*, vol. 75, no. 4, pp. 409–428, 2020.

[24] A. Ghasemi, V. Ghafouri, A. Barshan, and M. Rezaei, "Multi-objective traffic engineering in software-defined networks using metaheuristics," *IEEE Access*, vol. 10, pp. 64 210–64 225, 2022.

[25] S. Ayoubi, N. Zhang, C. E. Rothenberg, and C. Assi, "A survey on machine learning for software-defined networks: Data plane, control plane and management plane," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 3171–3209, 2018.

[26] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: Evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, p. 16, 2018.

[27] J. Xie, P. Dong, Z. Zhao, and H. Zhang, "Deep reinforcement learning-based traffic engineering for software-defined networks," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1895–1908, 2023.

[28] J. P. Versiani Ladeia and N. A. Maia, "Otimização aplicada ao problema de engenharia de tráfego em redes definidas por software," Master's thesis, Universidade Estadual de Montes Claros (Unimontes), Programa de Pós-Graduação em Modelagem Computacional e Sistemas (PPGMCS), Montes Claros, Brazil, 2018.

[29] C. Cascone, S. Vissicchio, and P. Bosshart, “Compiling packet processing pipelines for software-defined switches,” *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 182–196, 2020.